

My google colab note book link:

<https://colab.research.google.com/drive/1MQNaMpLzRhQvioaVt17Dx3LAh6ONf2Sr?usp=sharing>

## 1. Finding 4 Features that are used to model prediction:

a) bldgarea ( building area )

It is the best predictor such that whenever: bigger building→ more construction rate→ occupied more space→ higher price. This is the most demanding feature in Real estate..

b) unitsres (residential units )

It is the second driven factor such that whenever: more apartments in a building→per apartment rental value will increase gradually..

c) lotarea

It is the third demanding factor such that: properties built on larger land areas typically have higher value because land is a valuable resource in urban areas.

#### d) latitude or longitude

It is the valuable factor such that: it represents the geographical location of the property. location strongly influences property prices because houses in better areas tend to be more expensive..

## #Defining each step of linear regression model

a)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

I imported Pandas, NumPy, and Matplotlib because they are core libraries for machine learning. Pandas is used to load and manipulate the dataset, NumPy is used for efficient numerical and matrix operations required for machine learning algorithms, and Matplotlib is used to visualize the data through plots and graphs.

b)

```
df = pd.read_csv('nyc_housing_base.csv')
df.head()
```

The first line uses the pandas `read_csv()` function to load the NYC housing dataset from a CSV file and store it in a DataFrame named `df`. A DataFrame is a tabular structure that allows easy data analysis and manipulation. The second line `df.head()` displays the first five rows of the dataset so we can relate the data structure, column names, and verify that the dataset was loaded correctly.

c)

**Count** — Number of non-null values in the column

**Mean** —  
Average value of the column

**Std** —  
Standard deviation (how much the values vary from the mean)

**Min** — Smallest value in the column

**25%** —  
First quartile (25% of data lies below this value)

**50%** —  
Median (middle value of the dataset)

**75%** —  
Third quartile (75% of data lies below this value)

**Max** —

Largest value in the column

`df.describe()` in **Pandas** provides information for all numerical columns in a dataset. It helps us quickly understand the distribution of the data by showing metrics such as count, mean, standard deviation, minimum, maximum, and quartile values.

#### d) `df.info()`

It is used to get a quick overview of a dataset in pandas. It shows the number of rows, column names, data types of each column, non-null values, and memory usage. This helps us identify missing values and understand the structure of the dataset .

e) `df.isnull().sum()` is used to detect missing values in a dataset.

`isnull()` checks each cell and returns True if the value is missing.

Then `sum()` counts the number of True values in each column, giving the total number of null values per feature.

f) `df.duplicated()` identifies duplicate rows in the DataFrame and returns True or False for each row. The `.sum()` counts how many duplicate rows exist in the dataset.

g) The `df.drop_duplicates()` function removes duplicate rows from the dataframe. Duplicate records can bias the machine learning model and lead to incorrect analysis.

h)

```
df = df.drop(['borough_y'], axis=1)
df= df.drop(['bldgclass'], axis=1)
```

This line removes the column `borough_y` and `bldgclass` from the pandas dataframe. The `drop()` function is used to delete columns or rows, and `axis=1` specifies that we are dropping a column. In this dataset, `borough_y` and `bldgclass` contains redundant information and categorical feature respectively, so removed this feature during data preprocessing to avoid duplicate features and keep the dataset clean before training the model.

i)

```
plt.hist(df['zip_code'])
```

It simply represent the histogram plot of `zip_code` column

And each bar represents how many houses are belonging to a particular zip code

```
plt.title("Distribution of ZIP CODE")
```

It basically add a title to the plot that show the distribution of zip code

```
plt.xlabel("zip_code")
```

It labels the X- axis and represents different zip codes on the x-axis

```
plt.ylabel("Frequency")
```

It labels the Y- axis and represents how many properties belong to zip codes

```
plt.show()
```

It generates the plots

#as there are many plots such as 1) histogram plot, 2) box plot ,3) scattering plot etc that i have showing.

j) 

```
corr = df.corr(numeric_only=True)
```

`corr()` calculates the **correlation coefficient between all numerical features** in the dataset.

`numeric_only=True` ensures that **only numeric columns are used**, ignoring categorical features like zip codes or text.

```
corr['sale_price'].sort_values(ascending=False)
```

This line only select co relayed value for the specific target and shorts the co relation from higher order to lower order.

k) `x_train = df["yearbuilt"].values`

`y_train = df["sale_price"].values`

This code extracts the input feature and target variable from the dataset. The `yearbuilt` column is used as the independent variable (`x_train`) and `sale_price` is used as the dependent variable (`y_train`). The `.values` function converts the pandas column into a NumPy array so that it can be used in numerical computations such as gradient descent while training the linear regression model.

l) `def cost_function(x, y, w, b):`

`x` → input features

`y` → actual house prices

`w` → weight (slope)

`b` → bias (intercept)

This function calculates **the total prediction error of the model**.

```
m = len(x)
```

It represents the number of data points in the dataset

```
cost_sum = 0
```

This stores the sum of squared errors for all data points.

```
for i in range(m):
```

The loop process each training example one by one.

```
f = w * x[i] + b
```

Calculate the predicted value for the current inputs.

```
cost = (f - y[i]) ** 2
```

This calculate the difference between predicted value and actual value and square it to form a positive value.

```
cost_sum += cost
```

Add the error of the current datapoint into the total cost.

```
total_cost = (1/(2*m)) * cost_sum
```

It calculated average squared error

```
return total_cost
```



m) `def gradient_function(x, y, w, b):`

This calculate the gradient of cost function with respect to weights and bias.

```
m = len(x)
dc_dw = 0
dc_db = 0
```

Initialise the gradients to be zero.

```
for i in range(m):
    f = w * x[i] + b

    dc_dw += (f - y[i]) * x[i]
```

$f - y[i] \rightarrow$  prediction error

Partial derivative of cost function with respect to weights

mathematically =  $(f(x) - y) x$

```
dc_db += (f - y[i])
```

This is gradient with respect to bias.

```
dc_dw = (1/m) * dc_dw
dc_db = (1/m) * dc_db
```

Divide by m= [no.of](#) samples to get avg. gradients

```
return dc_dw, dc_db
```

n) `def gradient_descent(x, y, alpha, iterations):`

Alpha → learning rate (step size for updating parameters)

Iteration → number of times the algorithm updates the parameters

`w = 0`

`b = 0`

We start with initial guesses for the model parameters:

`for i in range(iterations):`

The algorithm runs for a fixed number iteration to improve parameters.

`dc_dw, dc_db = gradient_function(x, y, w, b)`

This function will calculate the partial derivative of cost function with respect to weight and bias.

`w = w - alpha * dc_dw`

`b = b - alpha * dc_db`

Update the parameter in the opposition of gradient to to reduce the cost function

`print(f"Iteration {i}: Cost {cost_function(x, y, w, b)}")`

This will print the cost value at each iteration.

```
return w, b
```

o) `learning_rate = 0.01`

This controls **how big a step we take while updating parameters (w and b)** during gradient descent.

```
iterations = 18
```

This defines **how many times the algorithm updates the parameters**

```
final_w, final_b = gradient_descent(x_train, y_train,  
learning_rate, iterations)
```

we are calling a function **gradient\_descent()** that: that takes `x_train` and `y_train` as input feature and target value , using the learning rate and iteration ,updating the linear model(  $y = wx + b$  ) then it returns the final value as : `final_w` and `final_b`

```
print(f"w: {final_w:.4f}, b: {final_b:.4f}")
```

This print the learned parameter of linear regression model up tp 4th decimal places as( `:.4f` )

p) `plt.scatter(x_train, y_train, label='Data Points')`

```
x_vals = np.linspace(min(x_train), max(x_train), 100)
```

**`np.linspace()` generates 100 evenly spaced values between the minimum and maximum of `x_train`.**

These values are used to draw a **smooth regression line** across the entire data range.

```
y_vals = final_w * x_vals + final_b  
plt.plot(x_vals, y_vals, color='red', label='Regression Line')
```

This will create a continuous line which represent the best fit line learned by the linear regression model.